Multilayer Perceptron (MLP)

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science & Engineering
Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Tp.HCM on August 23, 2025

Content

ltsach@hcmut.edu.vn
Thanh-Sach LE

Linear Regression and Logistic Regression: Recap

ltsach@hcmut.edu.vn
Thanh-Sach LE

Linear Regression: Single Output

- **Goal:** Predict one numeric target y from feature vector $\mathbf{x} \in \mathbb{R}^d$.
- **Model:**

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

- **Training:** Minimize Mean Squared Error (MSE).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Limitation

Captures only **linear** relationships between inputs and output. Cannot represent complex nonlinear patterns.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Linear Regression: Multiple Outputs

- **Goal:** Predict a vector of targets $\mathbf{y} \in \mathbb{R}^m$ from input $\mathbf{x} \in \mathbb{R}^d$.
- **Model:**

$$\hat{\mathbf{y}} = W^\top \mathbf{x} + \mathbf{b}, \quad W \in \mathbb{R}^{d \times m}, \ \mathbf{b} \in \mathbb{R}^m$$

- Training: minimize sum of squared errors across all output dimensions.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2$$

Limitation

Still restricted to **linear** mappings from inputs to outputs. Nonlinear dependencies remain unmodeled.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Logistic Regression: Binary (BCE)

- **Goal:** Binary classification ($y \in \{0, 1\}$) from $\mathbf{x} \in \mathbb{R}^d$.
- **Model:**

$$\hat{p} = P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Loss (Binary Cross-Entropy, BCE):**

$$L_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^{n} \Big(y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \Big)$$

- **Prediction:** $\hat{y} = \mathbf{1}\{\hat{p} \geq \tau\}$ (default $\tau = 0.5$).

Decision Boundary

$$\mathbf{w}^\top \mathbf{x} + b = 0$$

Linear (line/hyperplane) in feature space.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Softmax Regression: Multiclass (Categorical CE)

- **Goal:** Multiclass classification ($y \in \{1, \ldots, K\}$).
- **Model:**

$$\hat{p}_k = P(y = k \mid \mathbf{x}) = \frac{\exp(\mathbf{w}_k^\top \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w}_j^\top \mathbf{x} + b_j)}, \quad k = 1, \ldots, K$$

- **Loss (Categorical Cross-Entropy):**

$$L_{\text{CE}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} \mathbf{1}\{y_i = k\} \log \hat{p}_{i,k}$$

- **Prediction:** $\hat{y} = \arg\max_k \hat{p}_k$.

Decision Boundaries

Between class k and j:

$$(\mathbf{w}_k - \mathbf{w}_j)^\top \mathbf{x} + (b_k - b_j) = 0.$$

Piecewise-linear partitions of the input space.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Linear vs Logistic Regression

- **Linear regression:** predicts real-valued outputs.
- **Logistic regression:** predicts probabilities $\in [0, 1]$, thresholded for classification.
- Both are **linear models**: decision boundary is linear in feature space.
- Limitation: cannot capture complex nonlinear patterns.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Motivation for MLP

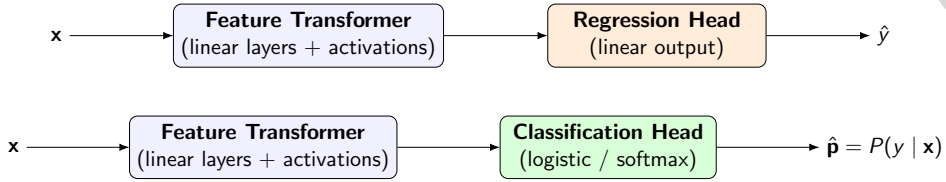
- Linear/Logistic regression: decision boundary is strictly **linear** in the input space.
- **MLP idea:** add hidden layers as **nonlinear feature transformers**.
- After transformation:
  - For regression: the output can be modeled by a linear function in feature space.
  - For classification: classes become more (or nearly) **linearly separable**.
- In essence: Deep learning = learn good nonlinear features + apply a linear head.

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Computational Architecture View

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Computational Architecture View

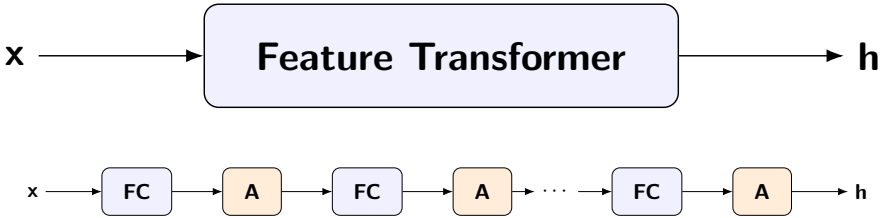


Notes

- **Feature Transformer**: learn nonlinear features from raw input $\mathbf{x}$.
- For **Regression**: the linear head outputs a single numeric prediction $\hat{y}$.
- For **Classification**: the head outputs class probabilities $\hat{\mathbf{p}}$ via logistic or softmax.
- Essence: Deep learning = nonlinear feature extraction + simple linear head.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Inside the Feature Transformer

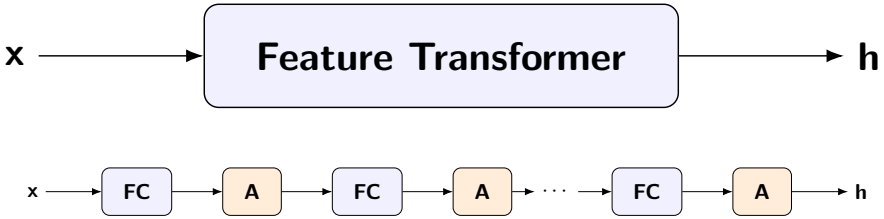


Notes

- **FC** = Fully Connected (Linear)
- **A** = Activation (Nonlinear)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Inside the Feature Transformer



Notes

- In MLP, the **Feature Transformer** is a stack of 0 to many pairs **[FC + A]**; each pair **[FC + A]** is called a **layer**.
- If 0 layers are used, then MLP reduces to a simple linear model:
  - Linear Regression (for regression head),
  - Logistic Regression (for binary classification head),
  - Softmax Regression (for multiclass classification head).

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Mathematical Model

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Forward Pass

- Each hidden layer computes:

$$\mathbf{h}^{(l)} = \phi\Big(W^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\Big), \quad l = 1, \ldots, L$$

- Input: $\mathbf{h}^{(0)} = \mathbf{x}$ (feature vector).
- $\phi(\cdot)$: nonlinear activation (ReLU, sigmoid, tanh, ...).
- Interpretation: alternating linear + nonlinear transforms build new features.

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Output Layer

Regression

$$\hat{y} = W^{(L+1)}\mathbf{h}^{(L)} + b^{(L+1)}$$

Classification (Softmax)

$$\hat{p}_k = \frac{\exp(\mathbf{w}_k^\top \mathbf{h}^{(L)} + b_k)}{\sum_j \exp(\mathbf{w}_j^\top \mathbf{h}^{(L)} + b_j)}, \quad k = 1, \ldots, K$$

- Final head = linear mapping from learned features.
- Regression: outputs real values, Classification: outputs class probabilities.

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP as Function Composition

- General form:

$$f(\mathbf{x};\theta) = f^{(L+1)} \circ \phi \circ f^{(L)} \circ \cdots \circ \phi \circ f^{(1)}(\mathbf{x})$$

- Each $f^{(l)}(\mathbf{u}) = W^{(l)}\mathbf{u} + b^{(l)}$.
- MLP = composition of linear transforms and nonlinear activations.
- **Key insight:** More layers $\Rightarrow$ higher representation power (ability to model complex nonlinear patterns).

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Layers

ltsach@hcmut.edu.vn
Thanh-Sach LE

Fully Connected (Linear) Layer — Math

- **Vector (single sample):** for $\mathbf{x} \in \mathbb{R}^N$, $W \in \mathbb{R}^{M \times N}$, $\mathbf{b} \in \mathbb{R}^M$

$$\mathbf{y} = W\mathbf{x} + \mathbf{b} \quad \in \mathbb{R}^M$$

- **Matrix (mini-batch):** for $X \in \mathbb{R}^{B \times N}$ (rows are samples)

$$Y = X W^\top + \mathbf{1}\mathbf{b}^\top \quad \in \mathbb{R}^{B \times M}$$

where $\mathbf{1} \in \mathbb{R}^B$ (broadcasting adds the bias to each row).

Remarks

- FC is a **linear mapping**; no nonlinearity is applied here.
- Stacking only FC layers (without activations) **collapses** to a single FC.
- Parameter count: $M \times N$ (weights) + M (biases).
- Bias can be absorbed by feature augmentation: $\tilde{\mathbf{x}} = [\mathbf{x}; 1]$, $\tilde{W} = [W\ \mathbf{b}]$.
- Common shapes: input (B, N), weight (M, N), output (B, M).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Fully Connected (Linear) Layer — Implementations

NumPy

```
import numpy as np
# Shapes: x: (N,), X: (B,N), W: (M,N), b: (M,)
y = W @ x + b                 # single sample
Y = X @ W.T + b               # mini-batch (broadcasts b over rows)
```

PyTorch

```
import torch
N, M = 128, 64
fc = torch.nn.Linear(N, M, bias=True)

x = torch.randn(32, N)        # batch of 32
y = fc(x)                     # shape: (32, M)

# Equivalent manual form:
y2 = x @ fc.weight.T + fc.bias
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Fully Connected (Linear) Layer — Implementations

Keras (TensorFlow)

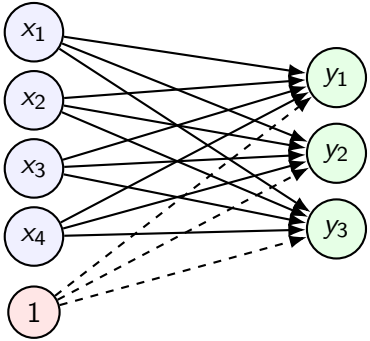
```
from tensorflow.keras import layers, models, Input

N, M = 128, 64
inp = Input(shape=(N,))
out = layers.Dense(M, use_bias=True)(inp)
model = models.Model(inp, out)
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Fully Connected (Linear) Layer — Diagram (N=4, M=3)

$$W \in \mathbb{R}^{3\times 4}, \quad \mathbf{b} \in \mathbb{R}^3$$



Each output neuron y_j: $\quad y_j = \sum_{i=1}^{4} W_{j,i} x_i + b_j.$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Activation: Sigmoid

Definition

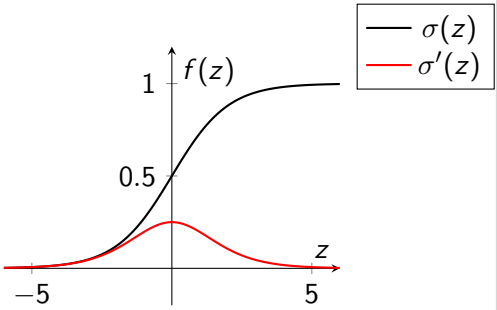
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

Notes

- Range $(0, 1)$, probabilistic interpretation.
- Saturates for large $|z| \Rightarrow$ vanishing gradients.
- Not zero-centered.

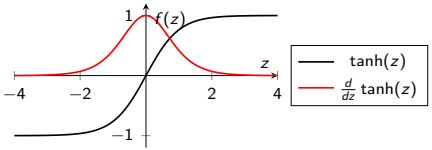


ltsach@hcmut.edu.vn
Thanh-Sach LE

Activation: Tanh

Definition

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Derivative

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$$

Notes

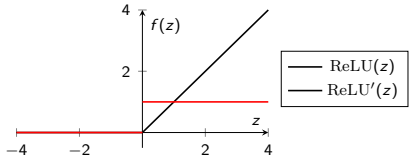
- Range $(-1, 1)$, zero-centered.
- Still saturates for large $|z|$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Activation: ReLU

Definition

$$\mathrm{ReLU}(z) = \max(0, z)$$



Derivative

$$\mathrm{ReLU}'(z) = \begin{cases} 0, & z < 0 \\ 1, & z > 0 \end{cases} \quad (\text{at } z = 0 \text{ set to 0 or 1})$$

Notes

- Simple, efficient; mitigates vanishing gradients.
- Risk of "dead neurons" ($z < 0$).

ltsach@hcmut.edu.vn
Thanh-Sach LE

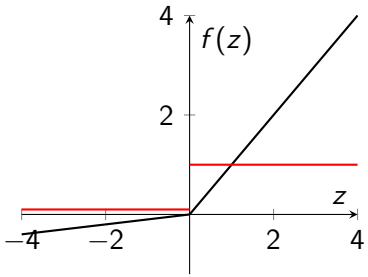
Activation: Leaky ReLU

Definition (with slope $\alpha \in (0, 1)$, e.g., 0.01)

$$\mathrm{LReLU}(z) = \begin{cases} z, & z \geq 0 \\ \alpha z, & z < 0 \end{cases}$$

Derivative

$$\mathrm{LReLU}'(z) = \begin{cases} 1, & z > 0 \\ \alpha, & z < 0 \end{cases}$$



Notes

- Avoids dead neurons by keeping a small negative slope.
- Behaves like ReLU when $\alpha \to 0$.

ltsach@hcmut.edu.vn
Thanh-Sach LE

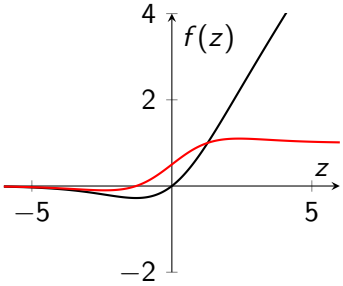
Activation: SiLU (Swish)

Definition

$$\mathrm{SiLU}(z) = z\,\sigma(z), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

Derivative

$$\frac{d}{dz}\big[z\,\sigma(z)\big] = \sigma(z) + z\,\sigma(z)\big(1-\sigma(z)\big)$$



Notes

- Smooth, non-monotonic; strong empirical performance.
- Widely used in modern architectures (e.g., ConvNets/Transformers variants).

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Summary

ltsach@hcmut.edu.vn
Thanh-Sach LE

MLP: Summary

- **Architecture View:** MLP = **Feature Transformer** (stack of [FC + Activation] layers) + a simple output head (regression or classification).
- **Mathematical Model:** Each hidden layer applies a linear mapping followed by a non-linear activation. Overall: composition of affine + nonlinear transformations.
- **Representation Power:** With enough hidden units, MLPs can approximate any continuous function (Universal Approximation Theorem).
- **Comparison to Linear/Logistic Regression:** Adds nonlinear feature transformation $\Rightarrow$ can model complex, nonlinear patterns.
- **Limitations:** Requires careful training (optimization, regularization). Prone to overfitting without sufficient data or regularization.

ltsach@hcmut.edu.vn
Thanh-Sach LE